02. 새로운 높이와 라운드 시작

목차

1. 새로운 높이 시작 과정
2. 새로운 라운드 진입
3. 검증자 우선순위 관리
4. 트랜잭션 대기 로직

새로운 높이 시작 과정

enterNewHeight 함수 (consensus/state.go)

새로운 블록 높이가 시작될 때 호출되는 핵심 함수입니다.

```
// Enter: `timeoutNewHeight` by startTime (commitTime+timeoutCommit),
// or, if SkipTimeoutCommit==true, after receiving all precommits from (height,r
func (cs *State) enterNewHeight(height int64, commitRound int32) {
// 높이 검증 및 상태 업데이트
// 새로운 높이에 대한 초기화 작업 수행
}
```

진입 조건

1. **timeoutCommit 만료**: 이전 블록 커밋 후 설정된 타임아웃 시간 경과
2. **SkipTimeoutCommit=true**: 모든 사전커밋을 받은 즉시 진행
3. **+2/3 사전커밋 수신**: 이전 높이에서 충분한 사전커밋 확보

주요 초기화 작업

- 라운드를 0으로 리셋
- 투표 집합 초기화
- 제안 관련 상태 클리어
- 잠금 상태 해제

새로운 라운드 진입

enterNewRound 함수 분석

```
func (cs *State) enterNewRound(height int64, round int32) {
logger := cs.Logger.With("height", height, "round", round)
// 1. 입력 검증
if cs.Height != height || round < cs.Round ||
(cs.Round == round && cs.Step != cstypes.RoundStepNewHeight) {
logger.Debug("entering new round with invalid args",
"current", log.NewLazySprintf("%v/%v/%v", cs.Height, cs.Round, cs.Step))
return
}
// 2. 시간 검증
```

```
if now := tmtime.Now(); cs.StartTime.After(now) {
logger.Debug("need to set a buffer and log message here for sanity",
"start_time", cs.StartTime, "now", now)
}
logger.Debug("entering new round",
"current", log.NewLazySprintf("%v/%v/%v", cs.Height, cs.Round, cs.Step))
}
```

검증자 우선순위 업데이트

```
// increment validators if necessary
validators := cs.Validators
if cs.Round < round {
validators = validators.Copy()
validators.IncrementProposerPriority(tmmath.SafeSubInt32(round, cs.Round))
}
```

우선순위 증가 로직

- **라운드 증가 시**: 검증자 우선순위를 라운드 차이만큼 증가
- **제안자 순환**: 우선순위에 따라 다음 제안자 선정
- **공정성 보장**: 모든 검증자가 순차적으로 제안 기회 획득

검증자 우선순위 관리

IncrementProposerPriority 함수 동작

검증자 집합에서 제안자를 공정하게 선택하기 위한 우선순위 시스템:

1. **우선순위 증가**: 각 검증자의 우선순위를 투표력에 비례하여 증가
2. **제안자 선택**: 가장 높은 우선순위를 가진 검증자를 제안자로 선정
3. **우선순위 감소**: 선택된 제안자의 우선순위를 전체 투표력만큼 감소

라운드 상태 업데이트

```
// Setup new round
cs.updateRoundStep(round, cstypes.RoundStepNewRound)
cs.Validators = validators

if round == 0 {
// 높이 0에서는 이미 리셋되었으므로 기존 제안 유지 가능
} else {
logger.Debug("resetting proposal info")
cs.Proposal = nil
cs.ProposalBlock = nil
cs.ProposalBlockParts = nil
}

cs.Votes.SetRound(tmmath.SafeAddInt32(round, 1)) // 다음 라운드도 추적
cs.TriggeredTimeoutPrecommit = false
```

이벤트 발행

```
if err := cs.eventBus.PublishEventNewRound(cs.NewRoundEvent()); err != nil {
cs.Logger.Error("failed publishing new round", "err", err)
}

cs.metrics.Rounds.Set(float64(round))
```

트랜잭션 대기 로직

WaitForTxs 설정

```
waitForTxs := cs.config.WaitForTxs() && round == 0 && !cs.needProofBlock(height)

if waitForTxs {
if cs.config.CreateEmptyBlocksInterval > 0 {
cs.scheduleTimeout(cs.config.CreateEmptyBlocksInterval, height, round,
cstypes.RoundStepNewRound)
}
} else {
cs.enterPropose(height, round)
}
```

조건별 동작

1. **WaitForTxs=true && round=0 && !needProofBlock:**

- 멤풀에 트랜잭션이 있을 때까지 대기
- `CreateEmptyBlocksInterval` 후 타임아웃으로 빈 블록 생성

2. **즉시 제안 진행 조건:**

- WaitForTxs=false
- round > 0 (재시도 라운드)
- needProofBlock=true (증명 블록 필요)

needProofBlock 함수

```
func (cs *State) needProofBlock(height int64) bool {
if height == cs.state.InitialHeight {
return true // 제네시스 블록 직후
}
lastBlockMeta := cs.blockStore.LoadBlockMeta(height - 1)
if lastBlockMeta == nil {
panic(fmt.Sprintf("needProofBlock: last block meta for height %d not found", hei
}
// 이전 블록이 앱 해시를 변경했는지 확인
return !bytes.Equal(lastBlockMeta.Header.AppHash, cs.state.AppHash)
}
```

트랜잭션 가용성 처리

```
func (cs *State) handleTxsAvailable() {
cs.mtx.Lock()
defer cs.mtx.Unlock()
// 라운드 0에서만 처리
if cs.Round != 0 {
return
}
switch cs.Step {
case cstypes.RoundStepNewHeight:
if cs.needProofBlock(cs.Height) {
return // enterPropose는 enterNewRound에서 호출됨
}
// RoundStepNewRound 타임아웃 스케줄링
timeoutCommit := cs.StartTime.Sub(tmtime.Now()) + 1*time.Millisecond
cs.scheduleTimeout(timeoutCommit, cs.Height, 0, cstypes.RoundStepNewRound)
case cstypes.RoundStepNewRound:
cs.enterPropose(cs.Height, 0)
}
}
```

타임아웃 처리

단계별 타임아웃 동작

```
switch ti.Step {
case cstypes.RoundStepNewHeight:
cs.enterNewRound(ti.Height, 0)
case cstypes.RoundStepNewRound:
cs.enterPropose(ti.Height, 0)
case cstypes.RoundStepPropose:
cs.enterPrevote(ti.Height, ti.Round)
case cstypes.RoundStepPrevoteWait:
cs.enterPrecommit(ti.Height, ti.Round)
case cstypes.RoundStepPrecommitWait:
cs.enterPrecommit(ti.Height, ti.Round)
cs.enterNewRound(ti.Height, ti.Round+1)
}
```

각 단계에서 타임아웃이 발생하면 다음 단계로 진행하거나 새로운 라운드를 시작합니다.

다음 문서에서는 제안(Propose) 단계의 상세한 동작을 살펴보겠습니다.